Revision — 1.4.1 Data Types

OCR H446 — one-page revision summary

Must know

- **Primitive types:** integer, real/float, char, string, Boolean. **Hex** is base 16 — **1 hex digit = 1 nibble = 4 bits**; 2 hex digits = 1 byte. Hex is a compact, lossless, error-reducing shorthand for binary (addresses, colour codes, machine code) — it does **not** reduce memory.
- **Conversions:** denary → binary — subtract largest place value that fits (or ÷2, read remainders bottom-up); binary → denary — add place values where bit = 1; binary → hex — split into nibbles **from the right**, convert each; hex → denary — $(\text{digit}_1 \times 16) + \text{digit}_0$.
- **Two's complement (signed):** 8-bit range **−128 to +127**; MSB place value is **−128** (MSB = 1 ⇒ negative). To negate: **invert ALL bits, then ADD 1** (e.g. +5 `00000101` → `11111010` → `11111011` = −5).
- **Subtraction:** A − B = A + (−B) — add the two's complement of B and **discard the final carry** out of the MSB.
- **Overflow:** result too large for the available bits — carry out of the MSB, or two same-sign numbers giving a wrong-sign result.
- **Shifts:** logical **left ×n** = **×2^n** (0s fill from right); logical **right ×n** = **÷2^n** unsigned (0s fill from left); **arithmetic right** copies the **sign bit** in from the left = **÷2^n signed**. Bits shifted off the end are **lost**; always state **direction AND effect**.
- **Floating point:** value = **mantissa × 2^exponent**. Mantissa = significant figures → **precision**; exponent = position of point → **range**. Both stored in two's complement.
- **Normalisation:** positive mantissa starts **`0.1…`**, negative starts **`1.0…`** (removes wasted leading bits → max precision). Moving point **right ⇒ exponent decreases**, **left ⇒ exponent increases** (value unchanged).
- **Trade-off (fixed total bits):** more mantissa = more **precision**, less range; more exponent = more **range**, less precision.
- **Character sets:** ASCII 7-bit = 128 chars (`'A'`=65, `'a'`=97, `'0'`=48). Unicode uses more bits (UTF-8 = 1–4 bytes) → ~1.1M code points for all languages + emoji; **ASCII is a subset** ⇒ backward-compatible.

Key terms

Term	Definition
Two's complement	Signed representation where MSB is a negative place value; negate = invert all bits then +1
Overflow	Result too large for the number of available bits
Mantissa	Significant-figures part of a float; sets **precision**
Exponent	Power-of-two part of a float; sets **range**
Normalisation	Adjusting a float so the mantissa starts with its first significant bit (`0.1…` +ve / `1.0…` −ve)
Logical shift	Shift filling with 0s; ×/÷ unsigned values by powers of 2
Arithmetic shift	Right shift preserving the sign bit; ÷ signed values by powers of 2
Character set	Mapping of characters to unique binary codes (e.g. ASCII, Unicode)

Worked example / how to

Calculate 45 − 28 in 8-bit two's complement (= 45 + (−28)): - +45 = `00101101` (32+8+4+1); +28 = `00011100` (16+8+4). - −28: invert `00011100` → `11100011`, then +1 → **`11100100`**. - Add: `00101101 + 11100100 = 1 00010001`. - **Discard the carry** out of the MSB → `00010001` = 16 + 1 = **17**. ✓ (45 − 28 = 17)

Exam tips

- Two's complement negation: invert **all** bits **then +1** — don't forget the +1; in subtraction **discard the final carry**.
- Shifts: always state **direction AND effect** (× or ÷ 2^n); arithmetic right keeps the **sign bit**; bits shifted off are lost.
- Normalisation: +ve starts `0.1`, −ve starts `1.0`; point **right ⇒ exponent down**, left ⇒ up. More mantissa = **precision**, more exponent = **range** (don't swap them).
- Hex does **not** save memory — it's a readable shorthand for binary; 1 hex digit maps exactly to 4 bits.